

Heart Disease Risk Prediction

An End-to-End MLOps Pipeline

Machine Learning Operations (MLOps) — AIMLCZG523

Assignment 01

Submitted by: Saurabh Gupta | Roll No: 2024AC05396

Data cleaning · EDA · model development · experiment tracking · CI/CD · containerization · deployment · monitoring

Heart Disease MLOps - End-to-End Architecture

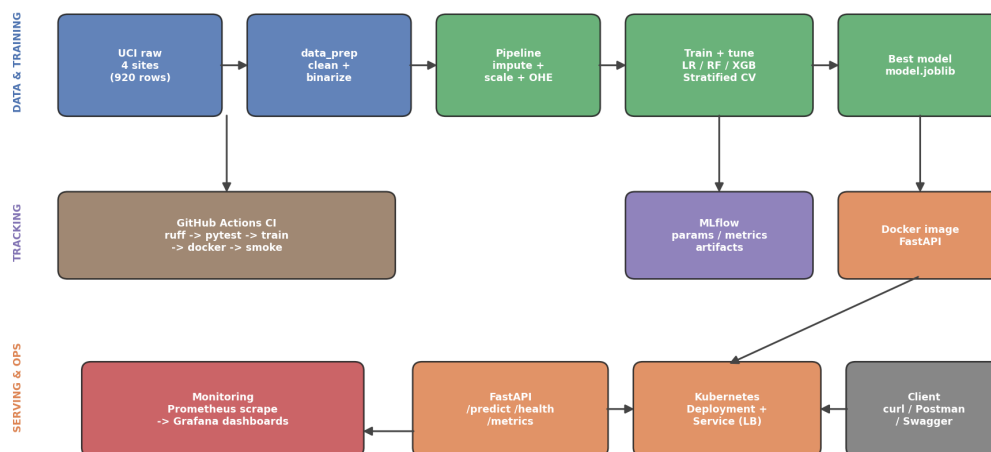


Table of Contents

- 1. Project Overview
 - 1.1 What makes this submission distinct
- 2. Setup and Installation
 - 2.1 Local setup
- 3. Dataset and Exploratory Data Analysis
 - 3.1 Class balance
 - 3.2 Missing values
 - 3.3 Feature distributions and correlations
 - 3.4 Prevalence by collection site
- 4. Feature Engineering and Preprocessing
- 5. Model Development and Evaluation
 - 5.1 Model comparison
 - 5.2 ROC and confusion matrix
- 6. Experiment Tracking with MLflow
- 7. Model Packaging and Reproducibility
- 8. CI/CD Pipeline and Automated Testing
 - 8.1 Test suite
- 9. Containerization
- 10. Production Deployment
- 11. Monitoring and Logging
- 12. Repository and Reproduction
- 13. Conclusion

1. Project Overview

This project builds and deploys a machine learning classifier that predicts the presence of heart disease from routine patient health data, then serves it as a containerized, monitored API. The goal was not only a model that scores well, but a pipeline that is reproducible end to end: anyone can clone the repository, install dependencies from a single requirements file, regenerate the cleaned data, retrain the model, run the test suite, and bring up the API in a container with one command.

The problem is framed as binary classification. Given thirteen clinical features (age, sex, chest pain type, resting blood pressure, cholesterol, and so on), the model outputs the probability that disease is present along with a discrete prediction and a low / moderate / high risk band. Because this is a screening context, recall (catching true cases) is treated as more important than raw accuracy, and that shaped how the final model was chosen and discussed.

1.1 What makes this submission distinct

Most public solutions to this dataset use the 303-row Cleveland subset alone. This project instead combines all four UCI collection sites — Cleveland, Hungary, Switzerland, and Long Beach VA — into one dataset of 920 patients. That choice has two benefits. It produces genuine, heavy missingness (the variables `ca`, `thal`, and `slope` are absent for large fractions of the non-Cleveland records), which forces a proper imputation strategy rather than a token one. And it makes the analysis and feature decisions individual rather than a copy of the standard Cleveland walkthrough.

2. Setup and Installation

The project targets Python 3.11. Every dependency is pinned in `requirements.txt`, and a `conda environment.yml` is provided as an alternative. The raw UCI files are committed under `data/raw`, so the pipeline runs offline and inside CI without depending on the UCI mirror.

2.1 Local setup

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python data/download_data.py --check      # verify raw files
python -m src.data_prep                   # build cleaned CSV
python -m src.eda                         # regenerate figures
python -m src.train                       # train, tune, log, save model
pytest -q && ruff check .                  # tests + lint
uvicorn api.main:app --reload              # serve at :8000/docs
```

A Makefile wraps each of these as a named target (`make install`, `make data`, `make train`, `make serve`, `make docker-build`, `make compose-up`), which keeps the common commands short and self-documenting.

3. Dataset and Exploratory Data Analysis

The cleaning layer (src/data_prep.py) concatenates the four site files, tags each row with its source, converts the dataset's "?" markers to true missing values, recodes a cholesterol of zero as missing (it is a not-measured placeholder, not a real reading), and collapses the original 0-4 severity target into a binary present / absent label. Imputation is deliberately not done here; it lives inside the model pipeline so fill values are learned only on training data.

3.1 Class balance

The combined dataset holds 509 positive and 411 negative cases, a 55 / 45 split. That is healthy enough that accuracy is informative, but precision and recall are still reported because a missed case is costly.

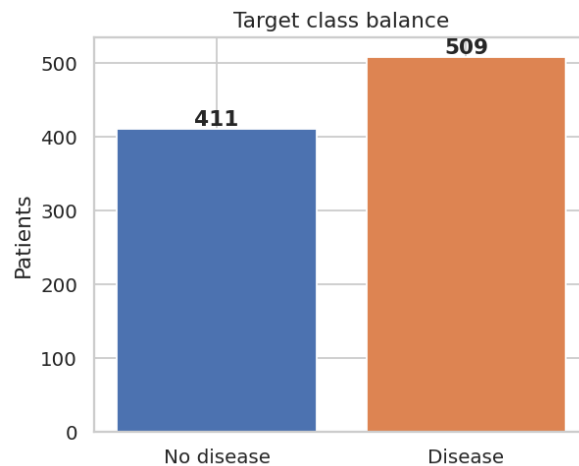


Figure 1. Target class balance across the combined dataset.

3.2 Missing values

Missingness is concentrated in ca (66%), thal (53%), and slope (34%), almost entirely from the non-Cleveland sites. This is the central data-quality challenge and the reason imputation is built into the pipeline rather than applied ad hoc.

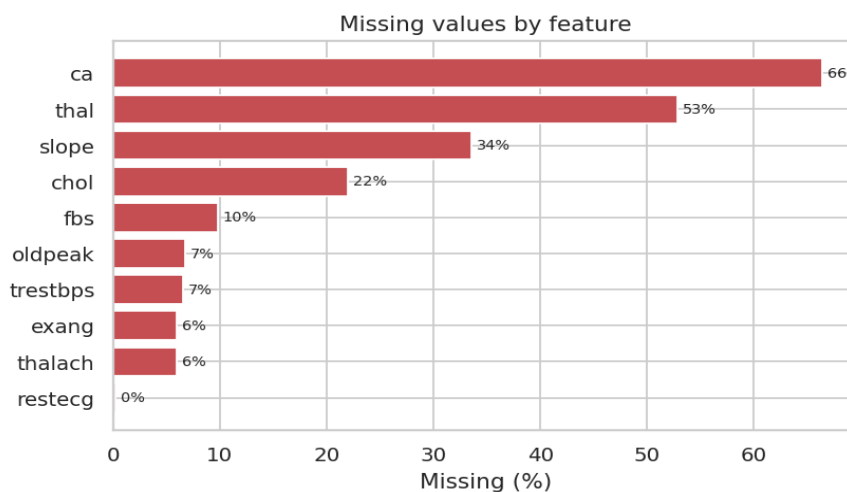


Figure 2. Percentage of missing values per feature.

3.3 Feature distributions and correlations

Overlaying the two classes on each numeric feature shows where they separate: max heart rate (thalach) and exercise-induced ST depression (oldpeak) shift clearly between groups, while cholesterol overlaps heavily. The correlation heatmap confirms the strongest individual correlates of the target are thal, cp, exang, and ca (positive) and thalach (negative). No two predictors are collinear enough to require dropping one.

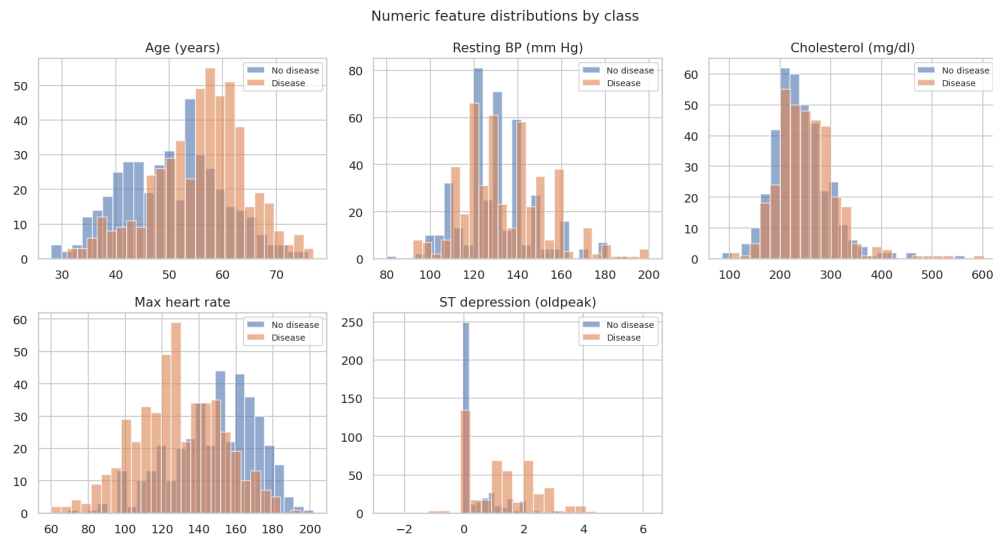


Figure 3. Numeric feature distributions, split by outcome.

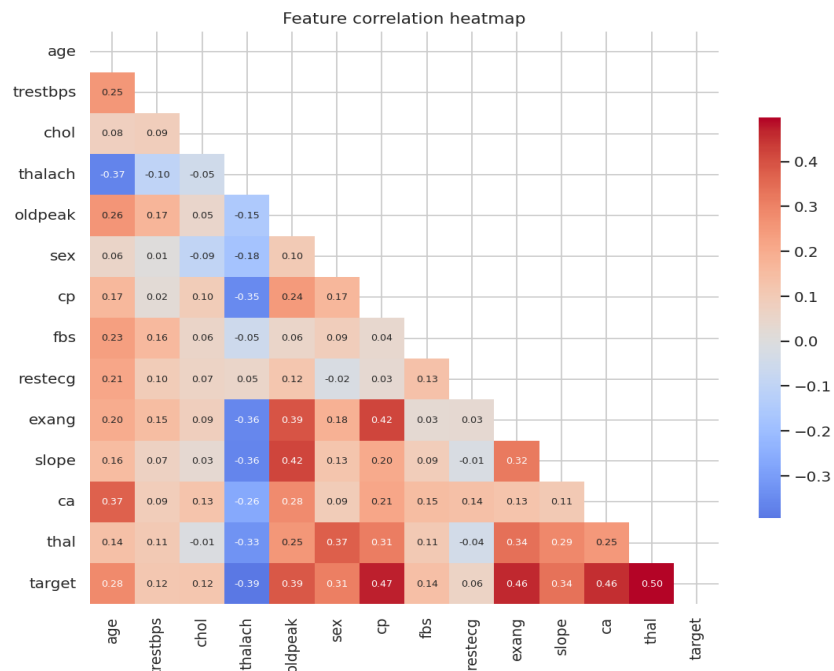


Figure 4. Feature correlation heatmap.

3.4 Prevalence by collection site

Disease prevalence ranges dramatically by site, from about 36% in Hungary to 94% in Switzerland. This is exactly why the source tag is excluded from the model features: leaving it in would let the model read the site label as a shortcut to prevalence instead of learning clinical signal, and would not generalize to a new hospital.

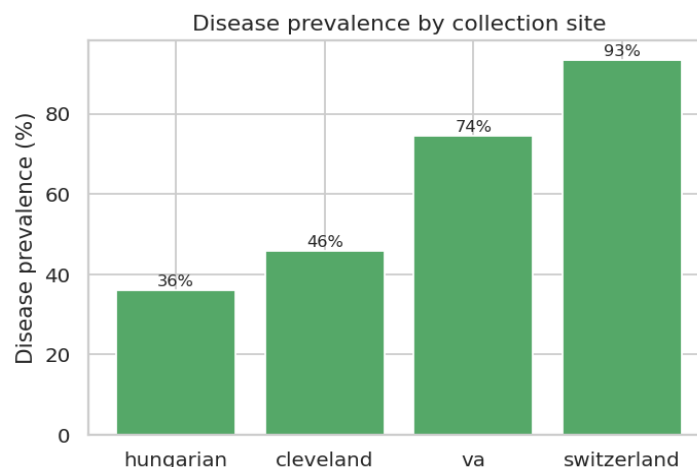


Figure 5. Disease prevalence per collection site.

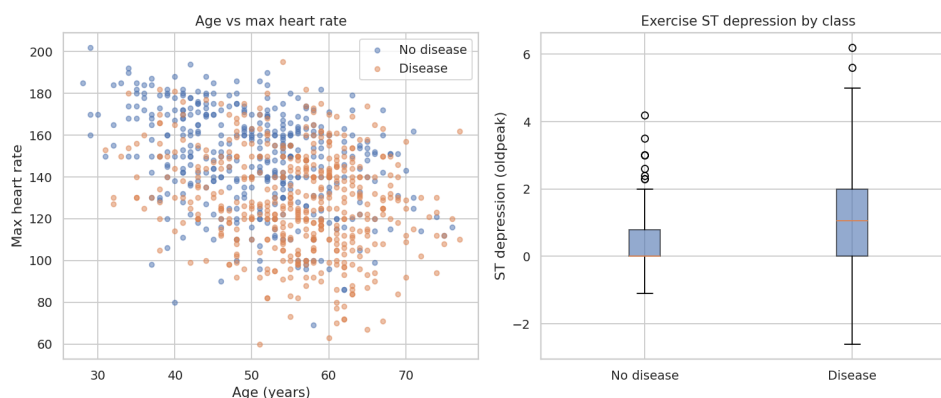


Figure 6. Age vs max heart rate, and ST depression by class.

4. Feature Engineering and Preprocessing

Preprocessing is implemented as a single scikit-learn ColumnTransformer wrapped in a Pipeline (src/preprocessing.py). Numeric features (age, trestbps, chol, thalach, oldpeak) are median-imputed and standardized. Categorical features (sex, cp, fbs, restecg, exang, slope, ca, thal) are imputed with the most frequent value and one-hot encoded with handle_unknown set to ignore, so an unseen category at inference does not crash the service.

Putting every transform inside the fitted pipeline is the key reproducibility decision. The median used to fill a missing cholesterol, the mean and standard deviation used to scale age, and the category set seen for chest pain type are all frozen inside the saved object. At inference, a raw patient record passes through identical maths to what the model saw in training, with no duplicated feature logic in the API.

5. Model Development and Evaluation

The data was split 80/20 with stratification on the target (736 train, 184 test). Four classifiers were trained: Logistic Regression (a linear baseline), Random Forest, XGBoost, and a calibrated SVM with an RBF kernel. Each was tuned with RandomizedSearchCV over 12-20 candidate configurations using 5-fold stratified cross-validation, optimizing ROC-AUC. Class weighting was enabled for the linear and tree-ensemble models to keep the minority class in focus.

5.1 Model comparison

All four models land in a tight band on ROC-AUC (0.90-0.91), so the tie-breaker was recall rather than raw accuracy, since this is a screening context where a missed positive case is the expensive error. On that basis XGBoost was selected as the production model: it reaches the highest recall (0.912) alongside the highest accuracy (0.848) of the four. Random Forest is a close second on ROC-AUC (0.910) with lower recall (0.843), and both linear baselines (Logistic Regression, SVM) trail on accuracy despite competitive AUC. The decision threshold could be lowered further to trade a little precision for additional recall if false negatives needed to be minimized even more aggressively.

Model	CV AUC	Test AUC	Accuracy	Recall	F1
Logistic Regression	0.881	0.895	0.799	0.824	0.820
Random Forest	0.883	0.909	0.815	0.843	0.835
XGBoost (selected)	0.879	0.905	0.848	0.912	0.869
SVM (RBF, calibrated)	0.883	0.900	0.793	0.863	0.822

The selected XGBoost model used 230 trees, max depth 3, a learning rate of 0.047, subsample ratio 0.801, column-subsample ratio 0.574, and L2 regularisation lambda of 2.09 — the shallow trees plus moderate learning rate combination that randomized search found best under cross-validation, consistent with a dataset this size favoring simpler, well-regularized trees over deep ones.

5.2 ROC and confusion matrix

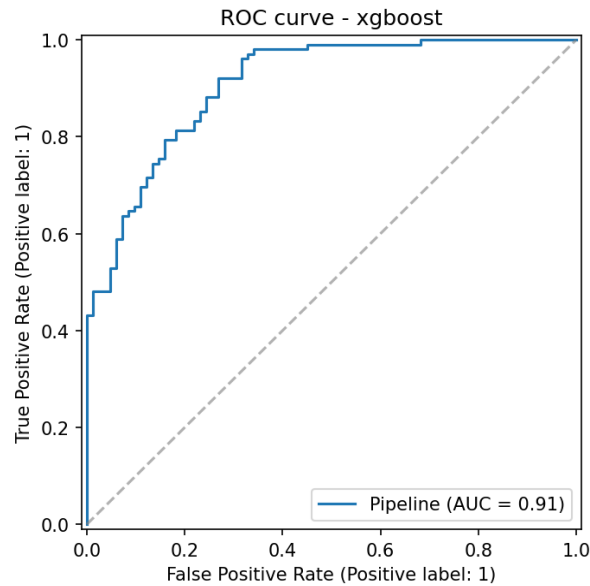


Figure 7. ROC curve for the selected XGBoost model on the held-out test set.

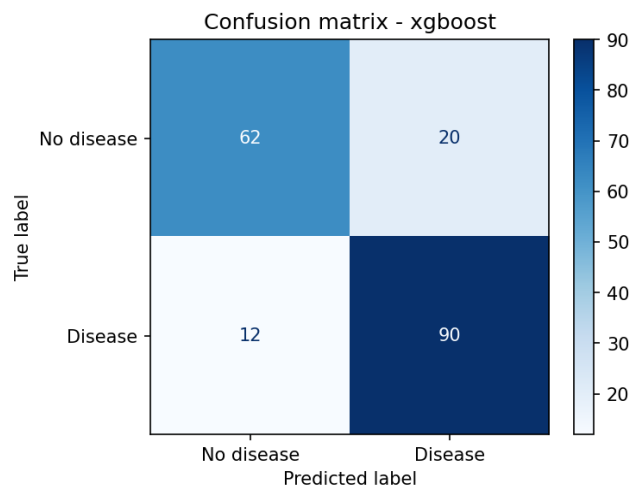


Figure 8. Confusion matrix for the selected XGBoost model.

6. Experiment Tracking with MLflow

Every training run is logged to MLflow under the experiment heart-disease-classification. For each model, the pipeline records the model type and the best hyperparameters, the cross-validated ROC-AUC, the full set of held-out test metrics (accuracy, precision, recall, F1, ROC-AUC), the fitted model itself, and two plot artifacts (ROC curve and confusion matrix). Because all four models log into one experiment, the MLflow UI gives a side-by-side comparison that makes model selection auditable rather than asserted.

The tracking store is a local `./mlruns` directory, browsable with `mlflow ui`. The CI workflow uploads this directory as a build artifact, so the experiment history for any commit is preserved alongside the run.

- **Parameters:** the model type and tuned hyperparameters

- **Metrics:** cv_roc_auc plus test accuracy, precision, recall, F1, and ROC-AUC
- **Artifacts:** the serialized pipeline, ROC curve, and confusion matrix per run

7. Model Packaging and Reproducibility

The winning pipeline is serialized to `models/model.joblib` with `joblib`, and a companion `model_metadata.json` captures the model type, training timestamp, feature schema, tuned parameters, train/test sizes, and the metrics for every candidate. The API reads this metadata at startup to report which model is live. Because the saved object is the entire pipeline, the preprocessing travels with the model and cannot drift.

Reproducibility rests on four supports: a single pinned `requirements.txt` (mirrored by `environment.yml`), a fixed random seed of 42 across splitting / CV / tuning, raw data committed in the repository, and a CI job that rebuilds everything from a clean checkout on every push.

8. CI/CD Pipeline and Automated Testing

Continuous integration runs on GitHub Actions (`.github/workflows/ci.yml`) on every push and pull request. The first job verifies dataset integrity, lints with `ruff`, builds the cleaned data, trains and selects the model, then runs the test suite, and uploads the model, EDA figures, and MLflow runs as artifacts. The second job builds the Docker image, runs the container, and executes a smoke test against the live endpoints. Any failing step — a lint violation, a broken test, or a training error — fails the pipeline and surfaces clear logs, satisfying the production-readiness requirement that the pipeline must stop on errors.

8.1 Test suite

The suite contains 22 tests across four files. They cover the data layer (site combination, target binarization, the chol-zero recode, missing-marker handling), the preprocessing pipeline (imputation leaves no NaN, one-hot expansion, unseen-category tolerance), the trained artifact (output schema, probability bounds, a minimum AUC floor), and the API (health, valid prediction, and four input-validation rejections returning HTTP 422).

Test file	Focus
<code>test_data_prep.py</code>	Loading, cleaning, target binarization
<code>test_preprocessing.py</code>	Imputation, encoding, pipeline fit/predict
<code>test_model.py</code>	Trained artifact and prediction helper
<code>test_api.py</code>	Endpoints and input validation

9. Containerization

The service is packaged with a multi-stage Dockerfile (`docker/Dockerfile`). The builder stage installs dependencies into a virtual environment; the slim runtime stage copies that environment plus the application code and the trained model, adds `libgomp1` for XGBoost and `curl` for the healthcheck, and runs as a non-root user. A `HEALTHCHECK` polls `/health` so the orchestrator knows when the container is ready.

```
docker build -t heart-disease-api:latest -f docker/Dockerfile .
docker run --rm -p 8000:8000 heart-disease-api:latest
bash scripts/smoke_test.sh # /health, /predict, /metrics
```

The API exposes POST /predict, which accepts a JSON patient record validated by a pydantic schema (each field range-checked against the UCI codebook) and returns the prediction, probability, confidence, and risk band. GET /health reports liveness and whether the model loaded; GET /metrics exposes Prometheus metrics; /docs serves Swagger UI for interactive testing.

10. Production Deployment

The container deploys to Kubernetes via either raw manifests (k8s/) or a Helm chart (k8s/helm/heart-api). The Deployment runs two replicas with CPU and memory requests and limits, plus liveness and readiness probes on /health. A LoadBalancer Service exposes the API, and an optional Ingress routes by host. Prometheus scrape annotations on the pods let an in-cluster Prometheus discover the /metrics endpoint automatically.

```
kubect1 apply -f k8s/deployment.yaml -f k8s/service.yaml
# or: helm install heart k8s/helm/heart-api
```

On Minikube, building the image into the cluster's Docker daemon and running minikube service heart-api exposes the endpoint locally. Deployment screenshots (pods running, service endpoint, a live /predict response) belong in the screenshots folder of the repository.

11. Monitoring and Logging

The API instruments every request with prometheus_client. Three metric families are exposed: a request counter labelled by endpoint, method, and status; a latency histogram per endpoint; and a counter of predictions by predicted class. A middleware times each request and writes a structured log line to stdout, which is the right place for logs inside a container so a log driver can collect them.

docker compose up brings up the API alongside Prometheus and Grafana. Prometheus scrapes /metrics every ten seconds; Grafana auto-provisions the Prometheus datasource and a Heart Disease API dashboard showing request rate, p95 latency, the split of predicted classes, and the 5xx error rate. In an ML context this matters because it surfaces API downtime, latency regressions, and shifts in the prediction mix that can hint at data drift.

12. Repository and Reproduction

The repository is organized by concern: api/ for the service, src/ for data and model code, tests/ for the suite, data/ for raw and processed data plus the download script, models/ for the saved artifact, docker/ and docker-compose.yml for containers, monitoring/ for Prometheus and Grafana config, k8s/ for manifests and the Helm chart, .github/workflows/ for CI, and reports/ for figures and this document.

Code repository: <https://github.com/guptasaurabh1/heart-disease-mlops>

To reproduce from scratch: clone the repository, create the environment, then run `make data`, `make train`, `make test`, and `make serve`. Building the cleaned data, training all four models with tuning, and saving the winner takes well under a minute on a laptop.

13. Conclusion

This project delivers a working heart-disease classifier wrapped in the full MLOps lifecycle. The selected XGBoost model reaches a recall of 0.912 and test ROC-AUC of 0.905 on a held-out split, chosen specifically for its recall advantage in this screening context, with Random Forest a close second on ROC-AUC (0.910). More importantly, the surrounding machinery — a reproducible preprocessing pipeline, MLflow tracking, a tested CI/CD workflow, a containerized API, Kubernetes manifests, and Prometheus/Grafana monitoring — turns a notebook model into something that can be retrained, validated, shipped, and watched in production. The natural steps ahead include automated data-drift detection feeding the monitoring stack and a scheduled retraining job triggered when incoming data diverges from the training distribution.